```
#!/usr/bin/env python3
"""ship-alert: AIS + GPS -> posiciones, CPA/TCPA y alertas de aproximación. 100%
import argparse, json, math, socket, threading, time
import pynmea2, serial
from pyais import decode as ais_decode

NM = 1852.0
own = {"lat": None, "lon": None, "sog": 0.0, "cog": 0.0, "ts": 0}
targets = {}            # mmsi -> dict
frag_buf = {}           # buffer multipart AIS
last_alert = {}         # mmsi -> timestamp
lock = threading.Lock()

def haversine(lat1, lon1, lat2, lon2):
    p1, p2 = math.radians(lat1), math.radians(lat2)
    a = math.sin((p2-p1)/2)**2 + math.cos(p1)*math.cos(p2)*math.sin(math.radians(
    return 2*6371000*math.asin(math.sqrt(a))

def bearing(lat1, lon1, lat2, lon2):
    p1, p2, dl = map(math.radians, (lat1, lat2, lon2-lon1))
    y = math.sin(dl)*math.cos(p2)
    x = math.cos(p1)*math.sin(p2) - math.sin(p1)*math.cos(p2)*math.cos(dl)
    return (math.degrees(math.atan2(y, x)) + 360) % 360

def vel(sog_kn, cog):
    v = sog_kn*0.514444
    return v*math.sin(math.radians(cog)), v*math.cos(math.radians(cog))  # este,

def cpa_tcpa(t):
    brg = math.radians(bearing(own["lat"], own["lon"], t["lat"], t["lon"]))
    d = haversine(own["lat"], own["lon"], t["lat"], t["lon"])
    rx, ry = d*math.sin(brg), d*math.cos(brg)
    ox, oy = vel(own["sog"], own["cog"]); tx, ty = vel(t["sog"], t["cog"])
    vx, vy = tx-ox, ty-oy
    vv = vx*vx + vy*vy
    if vv < 1e-6: return d, None
    tcpa = -(rx*vx + ry*vy)/vv
    if tcpa <= 0: return d, None
    return math.hypot(rx+vx*tcpa, ry+vy*tcpa), tcpa

def handle_ais(line):
    p = line.split(",")
    try:
```

```
        total, num, seq, ch = int(p[1]), int(p[2]), p[3], p[4]
    except (IndexError, ValueError): return
    key = (seq, ch)
    if total > 1:
        frag_buf.setdefault(key, {})[num] = line
        if len(frag_buf[key]) < total: return
        parts = [frag_buf[key][i] for i in sorted(frag_buf[key])]; del frag_buf[k
    else:
        parts = [line]
    try:
        d = ais_decode(*[x.encode() for x in parts]).asdict()
    except Exception: return
    m = d.get("mmsi")
    with lock:
        t = targets.setdefault(m, {"mmsi": m, "name": "", "lat": None, "lon": Non
        if d.get("shipname"): t["name"] = d["shipname"].strip("@ ").strip()
        la, lo = d.get("lat"), d.get("lon")
        if la is not None and lo is not None and abs(la) <= 90 and abs(lo) <= 180
            t["lat"], t["lon"] = la, lo
            sp, co = d.get("speed", 0), d.get("course", 0)
            t["sog"] = sp if sp is not None and sp < 102 else 0.0
            t["cog"] = co if co is not None and co < 360 else 0.0
            t["ts"] = time.time()

def handle_gps(line):
    try: msg = pynmea2.parse(line)
    except pynmea2.ParseError: return
    if msg.sentence_type == "RMC" and msg.status == "A":
        with lock:
            own.update(lat=msg.latitude, lon=msg.longitude, sog=msg.spd_over_grnd
                       cog=msg.true_course or own["cog"], ts=time.time())
    elif msg.sentence_type == "GGA" and msg.gps_qual:
        with lock: own.update(lat=msg.latitude, lon=msg.longitude, ts=time.time()

def handle_line(line):
    line = line.strip()
    if line.startswith(("!AIVDM", "!AIVDO", "!BSVDM")): handle_ais(line)
    elif line.startswith("$"): handle_gps(line)

def reader(src):
    """src: serial:/dev/ttyUSB0:38400 | udp:10110 | tcp:host:puerto"""
    kind, *rest = src.split(":")
    while True:
        try:
            if kind == "serial":
                with serial.Serial(rest[0], int(rest[1]) if len(rest) > 1 else 48
```

```
                    while True: handle_line(s.readline().decode(errors="ignore"))
            elif kind == "udp":
                s = socket.socket(socket.AF_INET, socket.SOCK_DGRAM); s.bind(("0.
                while True:
                    for ln in s.recv(4096).decode(errors="ignore").splitlines():
            elif kind == "tcp":
                s = socket.create_connection((rest[0], int(rest[1])), timeout=10)
                for ln in f: handle_line(ln)
        except Exception as e:
            print(f"[{src}] error: {e}. Reintentando en 5 s..."); time.sleep(5)

def loop(a):
    while True:
        time.sleep(a.interval)
        with lock:
            if own["lat"] is None:
                print("Esperando fix GPS..."); continue
            now = time.time(); rows = []
            for m, t in list(targets.items()):
                if t["lat"] is None or now - t["ts"] > a.timeout: targets.pop(m,
                d = haversine(own["lat"], own["lon"], t["lat"], t["lon"])
                cpa, tcpa = cpa_tcpa(t)
                rows.append((d, t, cpa, tcpa))
        rows.sort(key=lambda r: r[0])
        print(f"\n== Mi posición: {own['lat']:.5f}, {own['lon']:.5f} | {own['sog'
        for d, t, cpa, tcpa in rows[:a.top]:
            tc = f"{tcpa/60:.1f} min" if tcpa else "-"
            print(f"{t['mmsi']:>9} {t['name'][:20]:<20} {t['lat']:.4f},{t['lon']:
                  f"{d/NM:5.2f} nm  {t['sog']:4.1f} kn  CPA {cpa/NM:4.2f} nm  TCP
            peligro_dist = d/NM <= a.alert_nm
            peligro_cpa = tcpa is not None and cpa/NM <= a.cpa_nm and tcpa/60 <=
            if (peligro_dist or peligro_cpa) and time.time() - last_alert.get(t["
                last_alert[t["mmsi"]] = time.time()
                print(f"\a ⚠️ ALERTA: {t['name'] or t['mmsi']} a {d/NM:.2f} nm (
        with open(a.out, "w") as f:   # GeoJSON para cualquier visor/OpenCPN/QGIS
            json.dump({"type": "FeatureCollection", "features": [
                {"type": "Feature", "properties": {"mmsi": t["mmsi"], "name": t["
                 "geometry": {"type": "Point", "coordinates": [t["lon"], t["lat"]

if __name__ == "__main__":
    ap = argparse.ArgumentParser()
    ap.add_argument("--src", action="append", required=True, help="serial:/dev/tt
    ap.add_argument("--alert-nm", type=float, default=2.0, help="alerta si un bar
    ap.add_argument("--cpa-nm", type=float, default=0.5)
    ap.add_argument("--tcpa-min", type=float, default=20)
    ap.add_argument("--cooldown", type=int, default=120)
```

```
ap.add_argument("--timeout", type=int, default=600)
ap.add_argument("--interval", type=int, default=5)
ap.add_argument("--top", type=int, default=15)
ap.add_argument("--out", default="targets.geojson")
a = ap.parse_args()
for s in a.src: threading.Thread(target=reader, args=(s,), daemon=True).start
try: loop(a)
except KeyboardInterrupt: pass
```